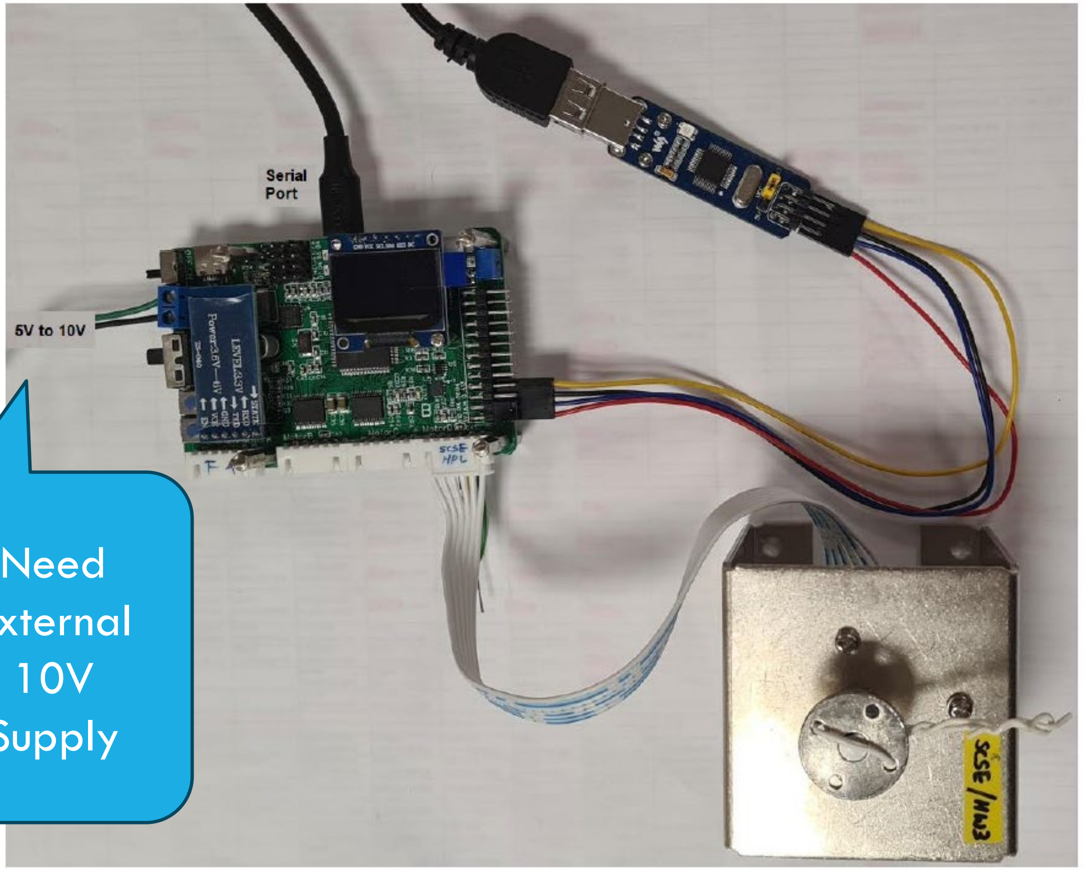
LAB 4 : BRIEFING

LAB 4: DIGITAL CONTROL OF DC MOTOR

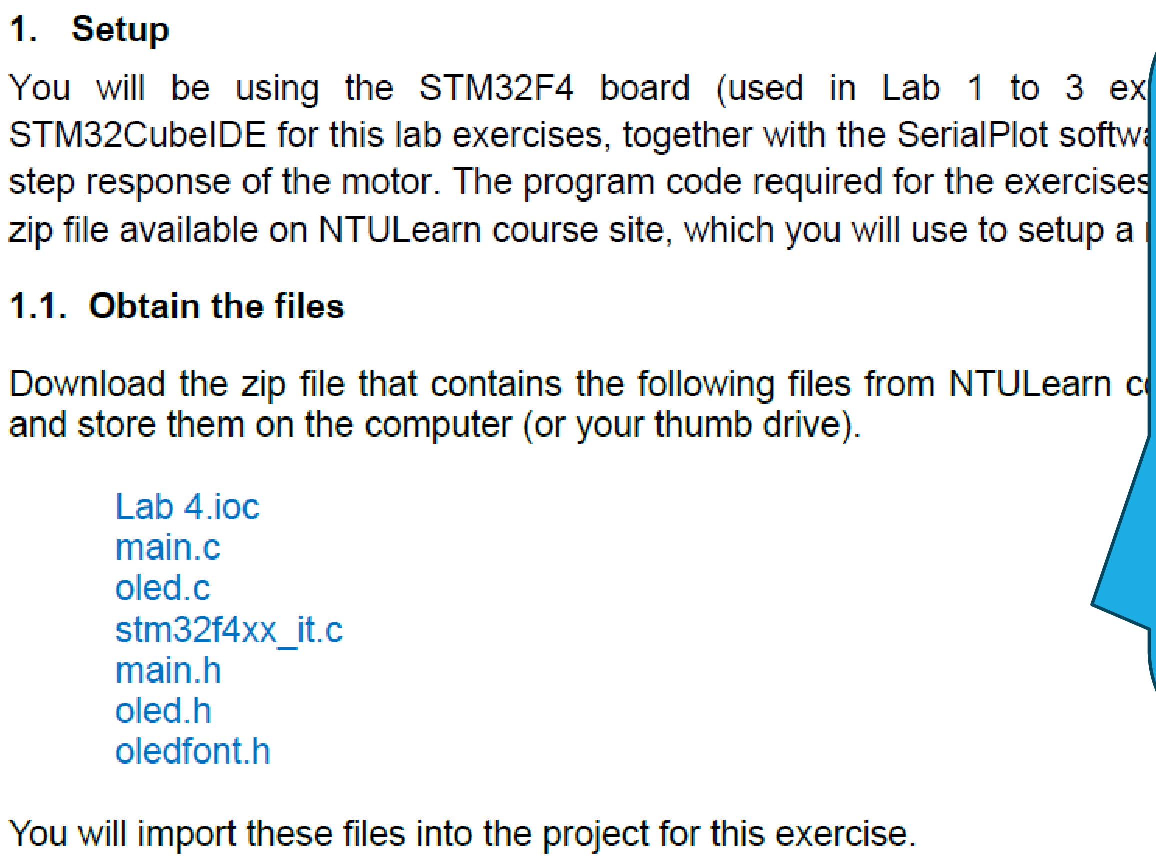

Serial Port
5V to 10V
SCSE HPL
SCSE/HW3

Need External 10V Supply

Equipment and accessories required

i) One desktop computer installed with STM32CubeIDE and SerialPlot.
ii) One STM32F4 board
iii) One ST-LINK SWD board
iv) One 20:1 gear DC Motor (attached with 26-poles/13 pulses Hall Effect Encoder)
v) External DC power supply (5V to 10V)

LAB 4: DIGITAL CONTROL OF DC MOTOR



1. Setup

You will be using the STM32F4 board (used in Lab 1 to 3 ex
STM32CubeIDE for this lab exercises, together with the SerialPlot softw
step response of the motor. The program code required for the exercises
zip file available on NTULearn course site, which you will use to setup a

1.1. Obtain the files

Download the zip file that contains the following files from NTULearn c
and store them on the computer (or your thumb drive).

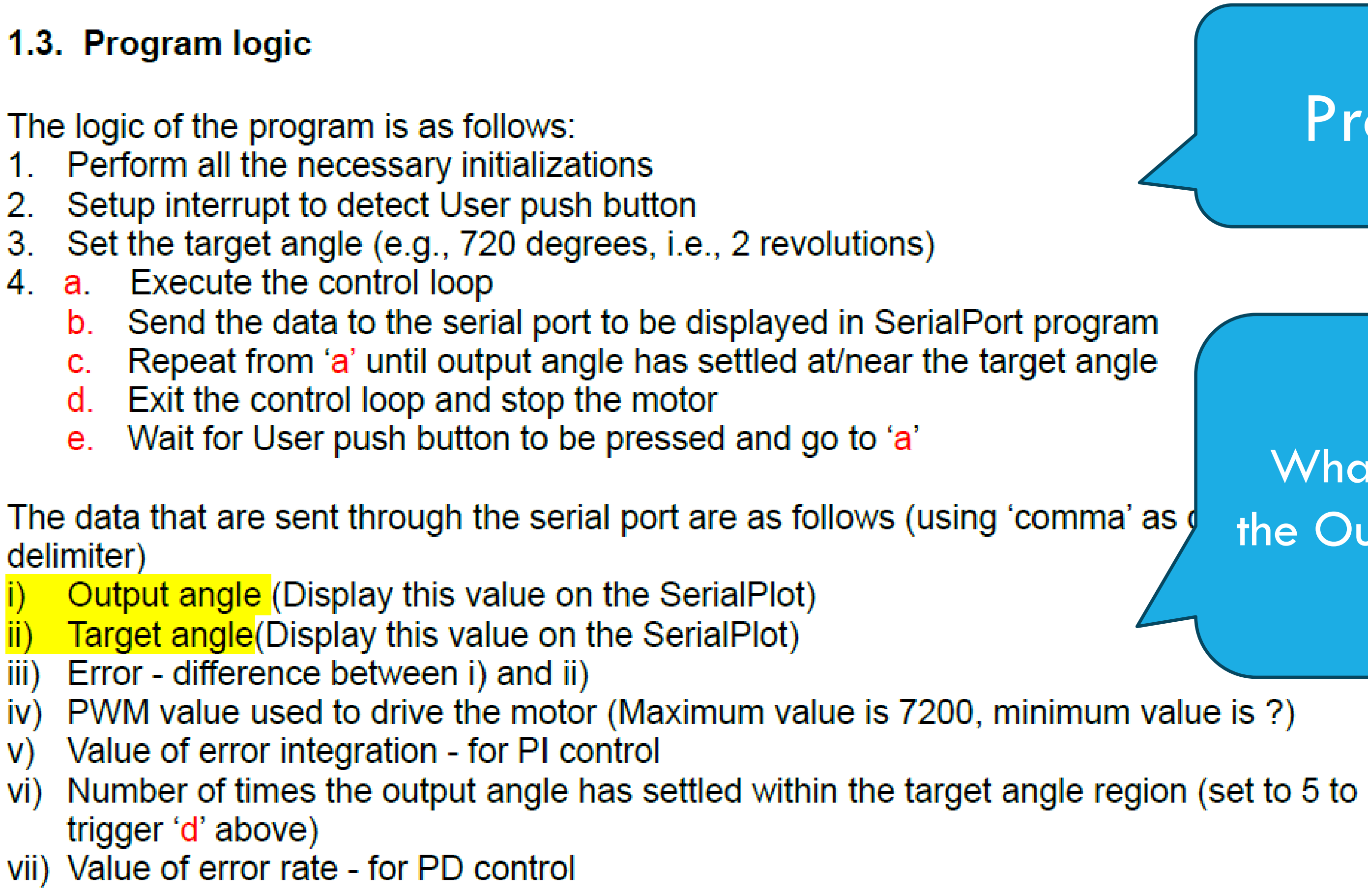
Lab 4.ioc
main.c
oled.c
stm32f4xx_it.c
main.h
oled.h
oledfont.h

You will import these files into the project for this exercise.

Good News :

Less Emphasis on the Programming Portion, Emphasis on the Understanding of the Control of DC Motor

LAB 4: DIGITAL CONTROL OF DC MOTOR



1.3. Program logic

The logic of the program is as follows:

1. Perform all the necessary initializations
2. Setup interrupt to detect User push button
3. Set the target angle (e.g., 720 degrees, i.e., 2 revolutions)
4. a. Execute the control loop
   b. Send the data to the serial port to be displayed in SerialPort program
   c. Repeat from 'a' until output angle has settled at/near the target angle
   d. Exit the control loop and stop the motor
   e. Wait for User push button to be pressed and go to 'a'

The data that are sent through the serial port are as follows (using 'comma' as
delimiter)

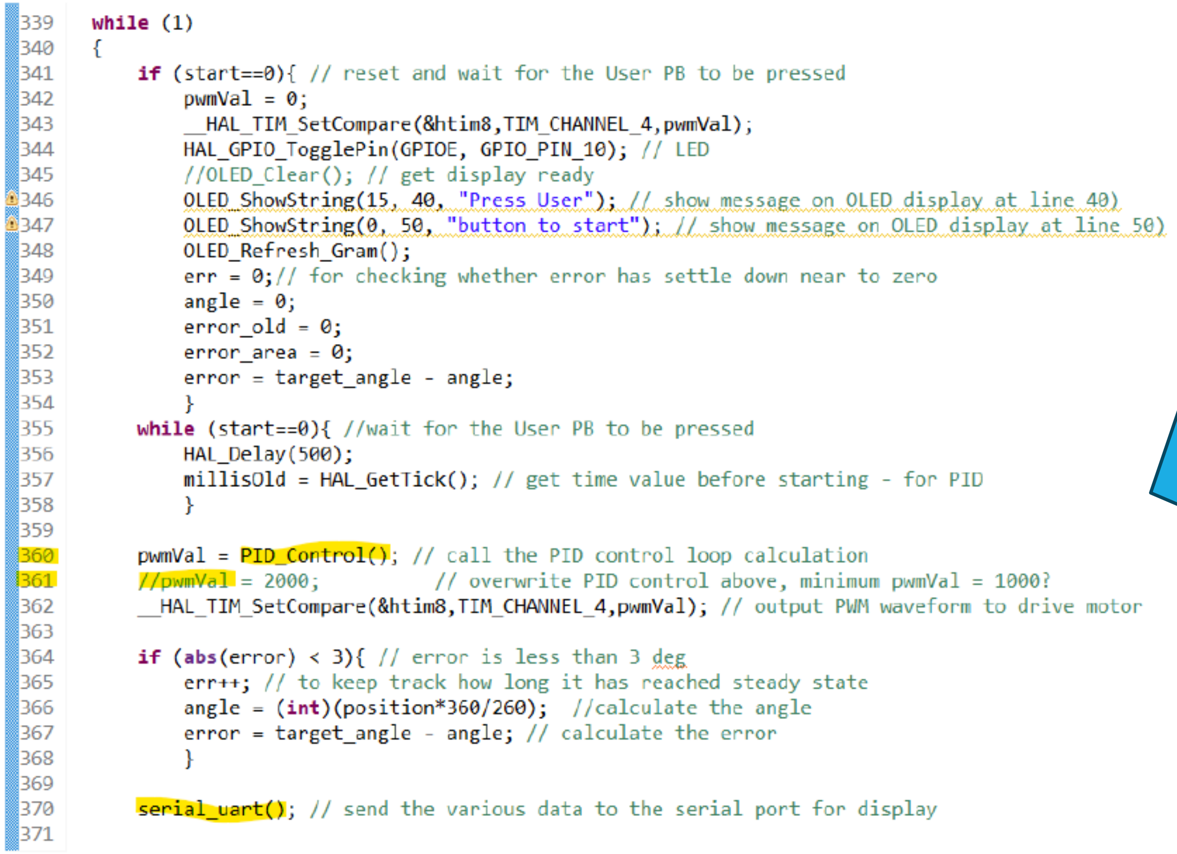
i) Output angle (Display this value on the SerialPlot)
ii) Target angle(Display this value on the SerialPlot)
iii) Error - difference between i) and ii)
iv) PWM value used to drive the motor (Maximum value is 7200, minimum value is ?)
v) Value of error integration - for PI control
vi) Number of times the output angle has settled within the target angle region (set to 5 to trigger 'd' above)
vii) Value of error rate - for PD control

Program Logic

What you can see from the Output of the software

LAB 4: DIGITAL CONTROL OF DC MOTOR

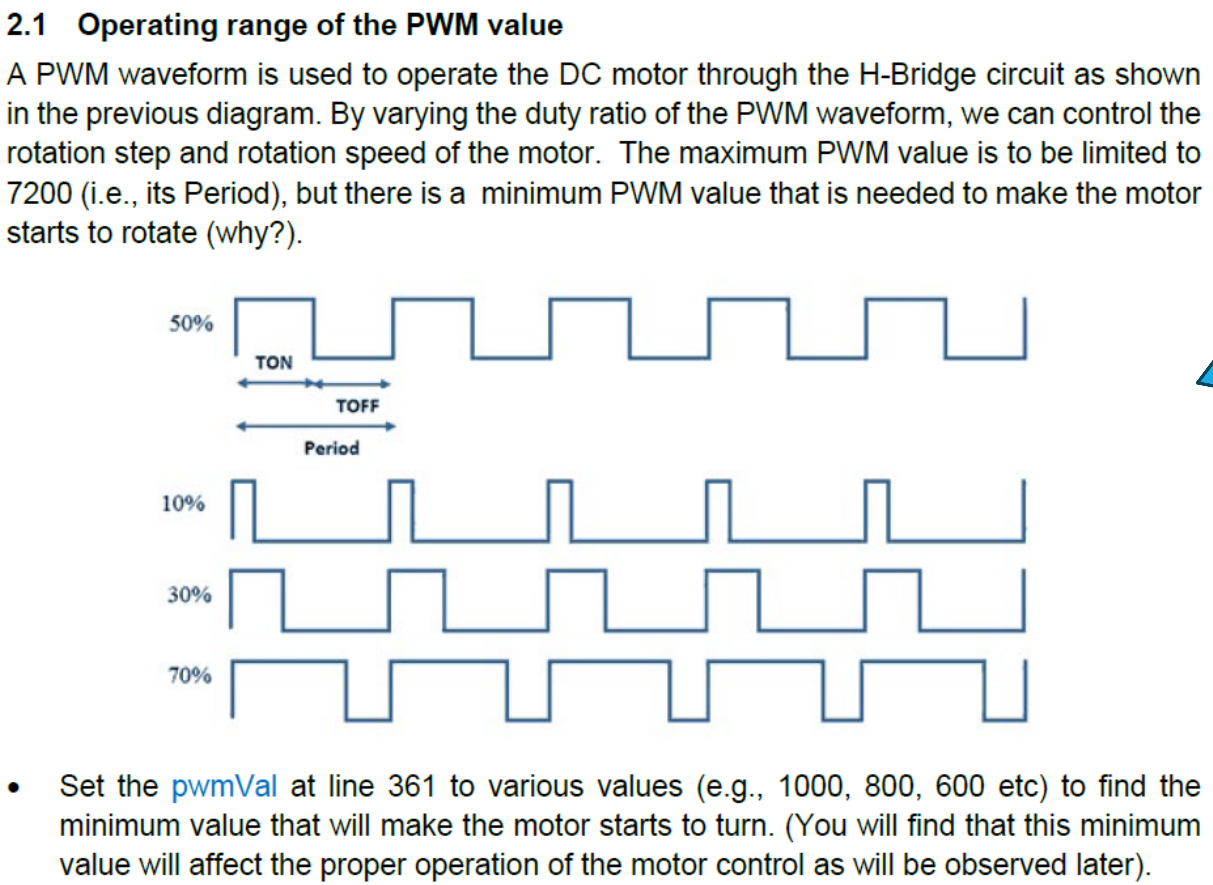
The PID control loop is called at line 360. But it can be ‘disabled’ by un-comment and preset the PWM (pwmVal) to a default value at line 361.



```
339  while (1)
340  {
341      if (start==0){ // reset and wait for the User PB to be pressed
342          pwmVal = 0;
343          __HAL_TIM_SetCompare(&htim8,TIM_CHANNEL_4,pwmVal);
344          HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_10); // LED
345          //OLED_Clear(); // get display ready
346          OLED_ShowString(15, 40, "Press User"); // show message on OLED display at line 40)
347          OLED_ShowString(0, 50, "button to start"); // show message on OLED display at line 50)
348          OLED_Refresh_Gram();
349          err = 0;// for checking whether error has settle down near to zero
350          angle = 0;
351          error_old = 0;
352          error_area = 0;
353          error = target_angle - angle;
354          }
355      while (start==0){ //wait for the User PB to be pressed
356          HAL_Delay(500);
357          millisOld = HAL_GetTick(); // get time value before starting - for PID
358          }
359
360      pwmVal = PID_Control(); // call the PID control loop calculation
361      //pwmVal = 2000;          // overwrite PID control above, minimum pwmVal = 1000?
362      __HAL_TIM_SetCompare(&htim8,TIM_CHANNEL_4,pwmVal); // output PWM waveform to drive motor
363
364      if (abs(error) < 3){ // error is less than 3 deg
365          err++; // to keep track how long it has reached steady state
366          angle = (int)(position*360/260);  //calculate the angle
367          error = target_angle - angle; // calculate the error
368          }
369
370      serial_uart(); // send the various data to the serial port for display
371
```

Spend time to understand the code (it will help you understand the experiment better)

LAB 4: DIGITAL CONTROL OF DC MOTOR



2.1 Operating range of the PWM value

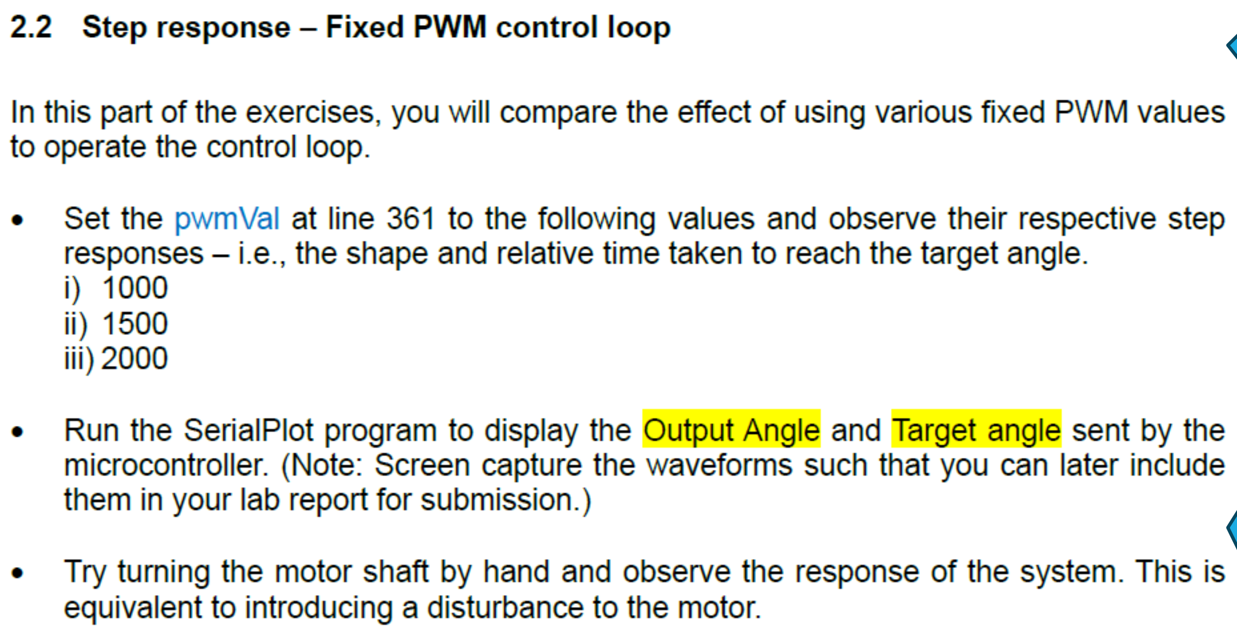
A PWM waveform is used to operate the DC motor through the H-Bridge circuit as shown in the previous diagram. By varying the duty ratio of the PWM waveform, we can control the rotation step and rotation speed of the motor. The maximum PWM value is to be limited to 7200 (i.e., its Period), but there is a minimum PWM value that is needed to make the motor starts to rotate (why?).

- Set the pwmVal at line 361 to various values (e.g., 1000, 800, 600 etc) to find the minimum value that will make the motor starts to turn. (You will find that this minimum value will affect the proper operation of the motor control as will be observed later).

Understand PWM mode of operation

Why is there a minimum value before the motor will move ?

LAB 4: DIGITAL CONTROL OF DC MOTOR



2.2 Step response – Fixed PWM control loop

In this part of the exercises, you will compare the effect of using various fixed PWM values to operate the control loop.

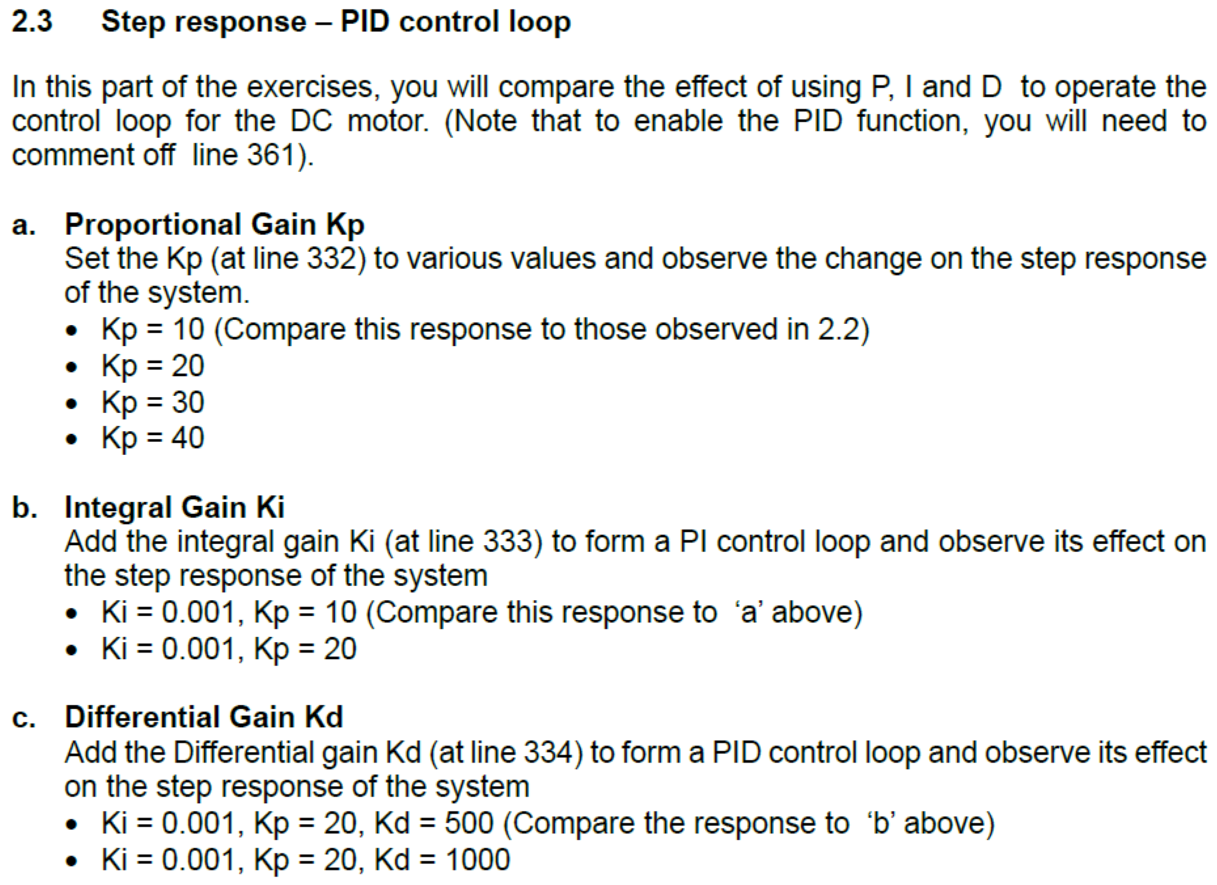
- Set the pwmVal at line 361 to the following values and observe their respective step responses – i.e., the shape and relative time taken to reach the target angle.
  i) 1000
  ii) 1500
  iii) 2000
- Run the SerialPlot program to display the Output Angle and Target angle sent by the microcontroller. (Note: Screen capture the waveforms such that you can later include them in your lab report for submission.)
- Try turning the motor shaft by hand and observe the response of the system. This is equivalent to introducing a disturbance to the motor.

Understand that Position Control has to be on a Closed-Loop System (explained later)

Every Iteration of this Code will move the Position of the Motor closer to the Target Angle.

It is quite obvious that a larger PWM value will move the position more in each iteration

LAB 4: DIGITAL CONTROL OF DC MOTOR



2.3 Step response – PID control loop

In this part of the exercises, you will compare the effect of using P, I and D to operate the control loop for the DC motor. (Note that to enable the PID function, you will need to comment off line 361).

a. **Proportional Gain Kp**

Set the Kp (at line 332) to various values and observe the change on the step response of the system.

- Kp = 10 (Compare this response to those observed in 2.2)
- Kp = 20
- Kp = 30
- Kp = 40

b. **Integral Gain Ki**

Add the integral gain Ki (at line 333) to form a PI control loop and observe its effect on the step response of the system

- Ki = 0.001, Kp = 10 (Compare this response to 'a' above)
- Ki = 0.001, Kp = 20

c. **Differential Gain Kd**

Add the Differential gain Kd (at line 334) to form a PID control loop and observe its effect on the step response of the system

- Ki = 0.001, Kp = 20, Kd = 500 (Compare the response to 'b' above)
- Ki = 0.001, Kp = 20, Kd = 1000

Proper Control of the DC Motor. In this Exercise, we will be looking at a PID Controller

Details of the Control Theory portion will follow in the next few slides

CONTROL THEORY :

Hours of Control Lectures summaries here in an hour.

SYSTEM MODELLING

1st Order System

$$x(t) = m\frac{\partial yt)}{\partial t} + ny(t)$$

$$H(s) = \frac{a}{s+b}$$

2nd Order System

$$x(t) = p\frac{\partial^2 y(t)}{\partial t^2} + q\frac{\partial y(t)}{\partial t} + ry(t)$$

$$H(s) = \frac{a}{s^2+bs+c}$$

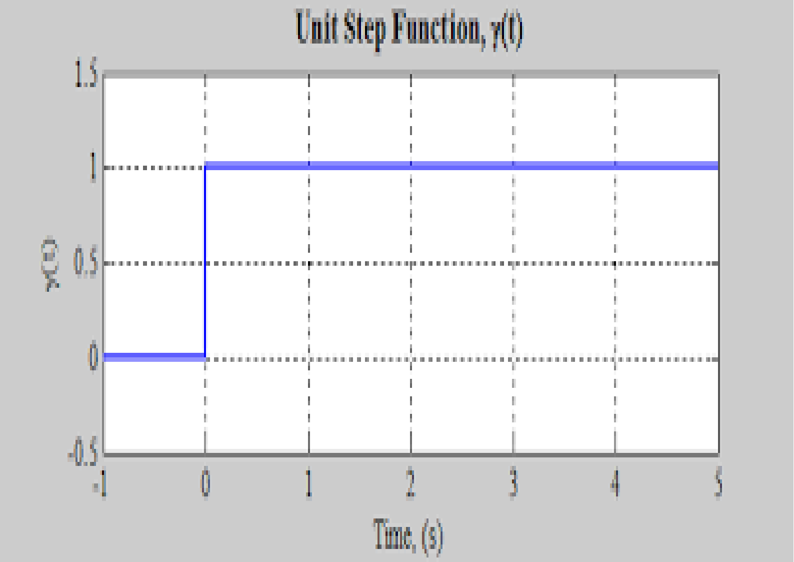
3rd Order System

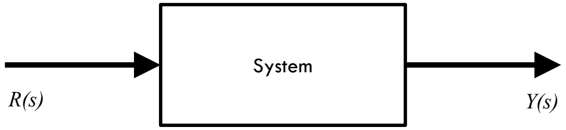
$$x(t) = u\frac{\partial^3 y(t)}{\partial t^3} + v\frac{\partial^2 y(t)}{\partial t^2} + w\frac{\partial y(t)}{\partial t} + zy(t)$$

$$H(s) = \frac{g}{s^3+ds^2+es+f}$$

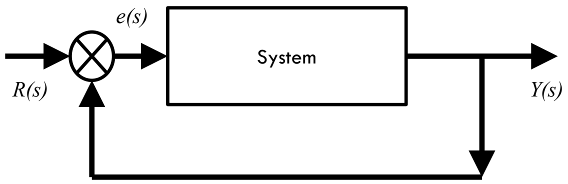
Position Control of DC Motor

STEP RESPONSE




System
R(s)
Y(s)

Transfer Function : $G(s) = \frac{Y(s)}{R(s)}$

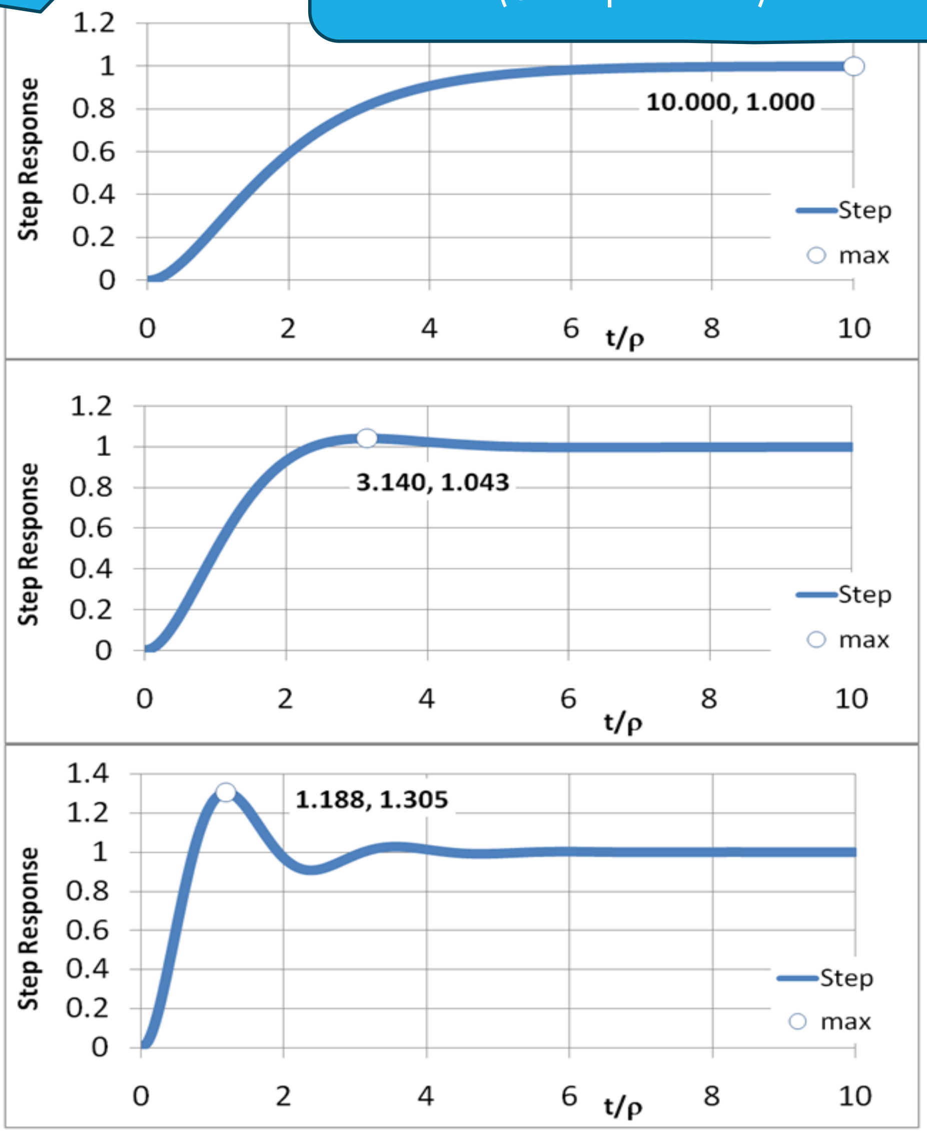

System
e(s)
R(s)
Y(s)

Transfer Function : $T(s) = \frac{G(s)}{1 + G(s)}$

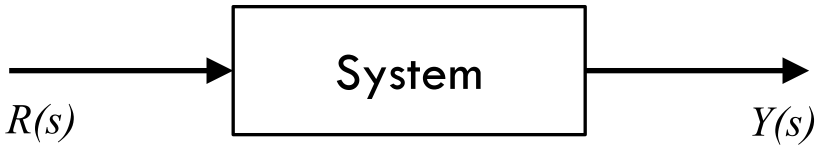
1st Order System
(1 unique case)

2nd Order System
(3 unique cases)

3rd Order System
(5 unique cases)



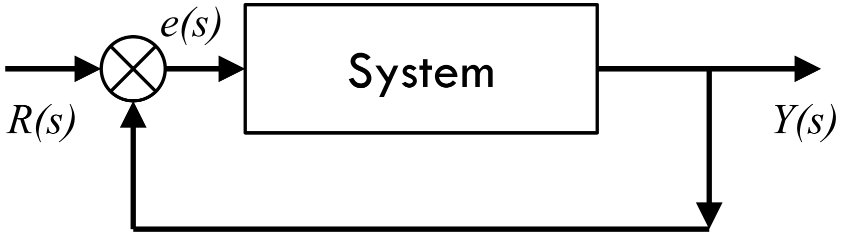
POSITION CONTROL


System
$R(s)$
$Y(s)$

Transfer Function :

$$G(s) = \frac{Y(s)}{R(s)}$$

Open Loop System


$e(s)$
System
$R(s)$
$Y(s)$

Transfer Function :

$$T(s) = \frac{G(s)}{1 + G(s)}$$

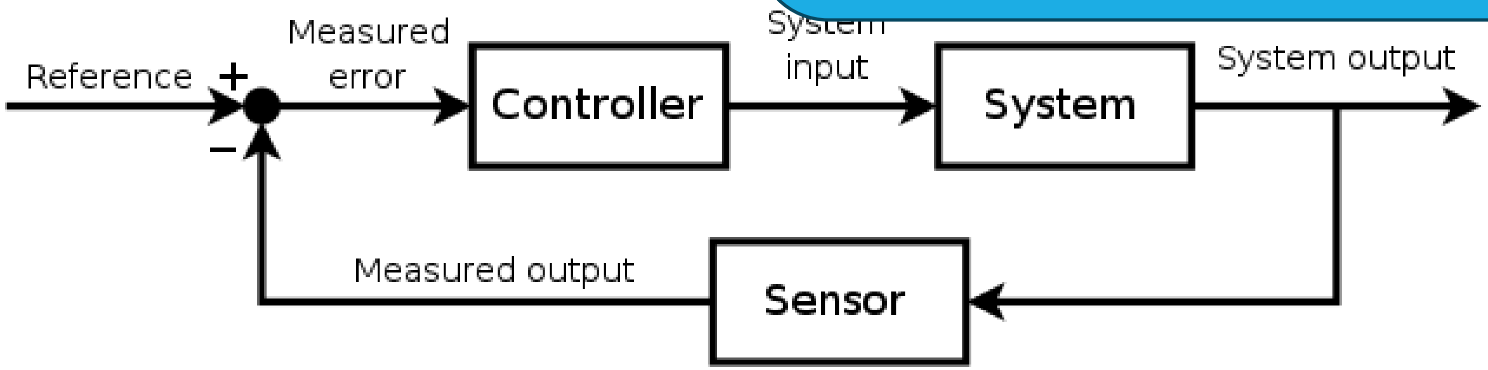
Closed Loop Unity Gain Negative Feedback System

CONTROLLERS

Controller

(with a proper design)

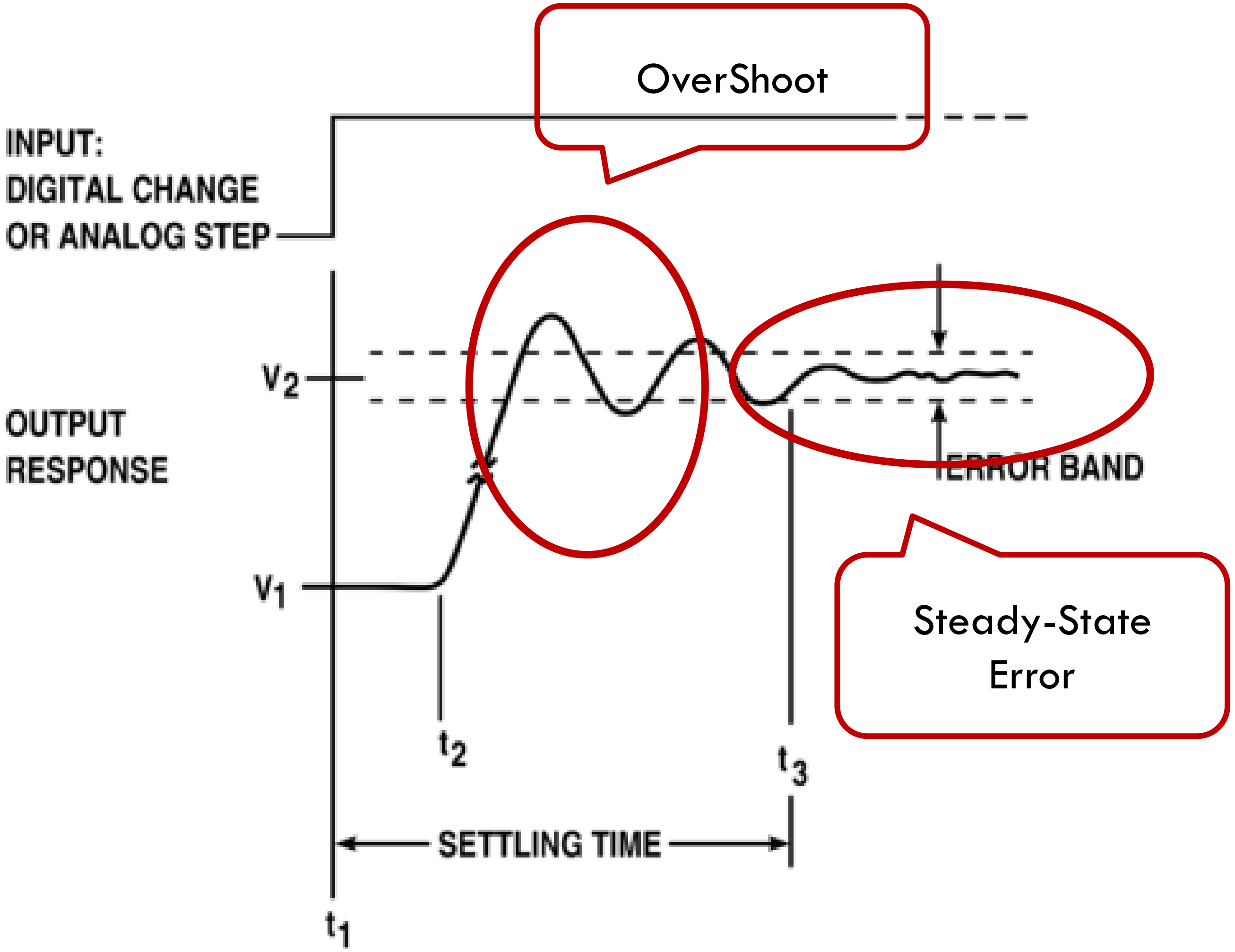
can be used to tune the step response to any of the preferred cases (response)


Reference
+
−
Measured error
Controller
System input
System
System output
Measured output
Sensor

Overall Transfer Function

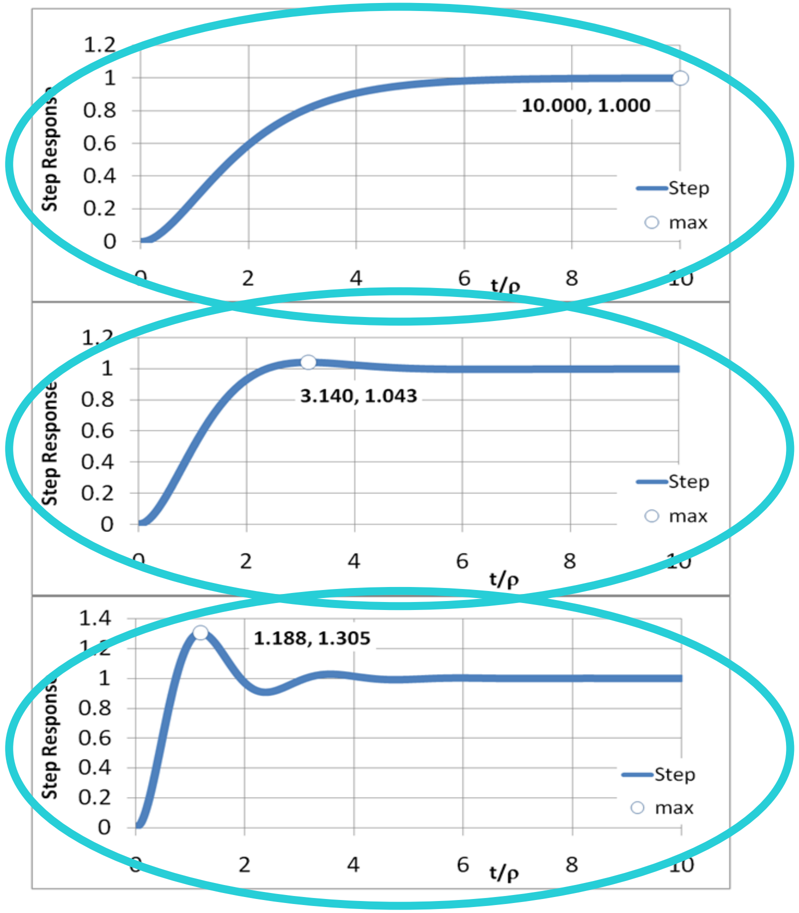
$$\frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{(1+C(s)G(s))}$$

CONTROLLER

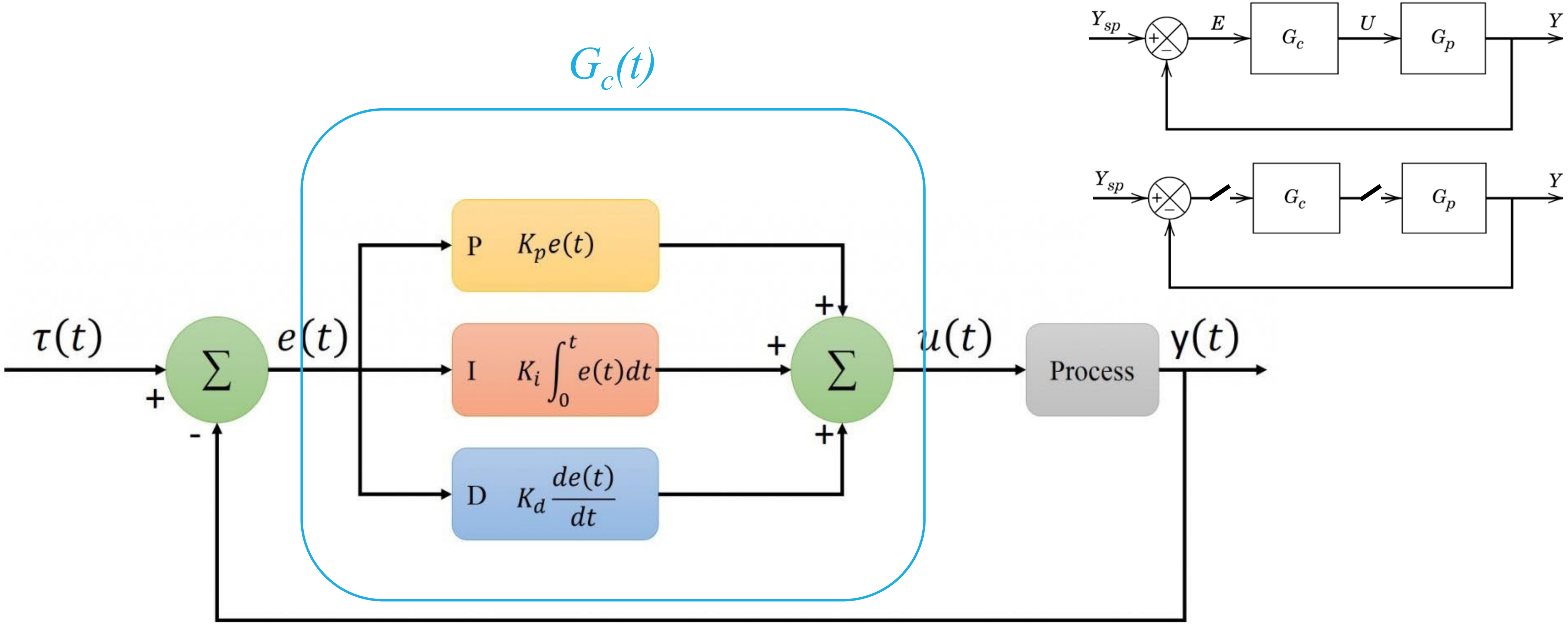

INPUT:
DIGITAL CHANGE
OR ANALOG STEP
OUTPUT
RESPONSE
V_2
V_1
ERROR BAND
t_2
t_3
SETTLING TIME
t_1

OverShoot

Steady-State Error



PID CONTROLLER GENERIC DESIGN



$G_c(t)$
$\tau(t)$
+
-
Σ
$e(t)$
P $K_p e(t)$
I $K_i \int_0^t e(t)dt$
D $K_d \frac{de(t)}{dt}$
+
+
+
Σ
$u(t)$
Process
$y(t)$
Y_{sp}
+
−
E
G_c
U
G_p
Y
Y_{sp}
+
−
G_c
G_p
Y

PID CONTROLLER GENERIC DESIGN

- Analog PID Implementation:

$$G_c(t) = K_P * e(t) + K_I * \int_0^t e(\tau)d\tau + K_D * \frac{de(t)}{dt}$$

Reduce the rise time

Reduce a steady offset from set point

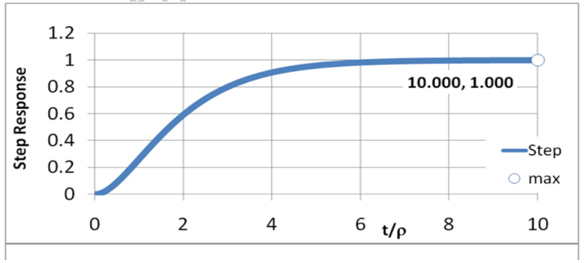
Reduce the overshoot

PID CONTROLLER GENERIC DESIGN

- Analog PID Implementation:

$$G_c(t) = \boxed{K_P * e(t)} + K_I * \int_0^t e(\tau)d\tau + K_D * \frac{de(t)}{dt}$$

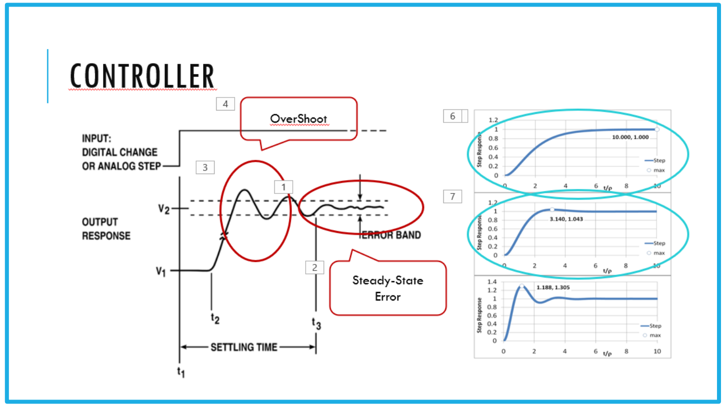
K_P value is small



K_P value is larger

K_P value is much larger

PID CONTROLLER GENERIC DESIGN


CONTROLLER

4
OverShoot
INPUT:
DIGITAL CHANGE
OR ANALOG STEP
3
1
V_2
OUTPUT
RESPONSE
ERROR BAND
V_1
2
Steady-State
Error
t_2
t_3
SETTLING TIME
t_1
6
7
Step Response
10.000, 1.000
3.140, 1.043
1.188, 1.305
Step
max
t/ρ
0
2
4
6
8
10
0.2
0.4
0.6
0.8
1
1.2
1.4

- Analog PID Implementation:

$$G_c(t) = K_P * e(t) + K_I * \int_0^t e(\tau)d\tau + \boxed{K_D * \frac{de(t)}{dt}}$$

Differentiation :

$$K_D * \frac{dy(t)}{dt}$$

if y(t) = constant, differentiate $\frac{dy(t)}{dt} = 0$

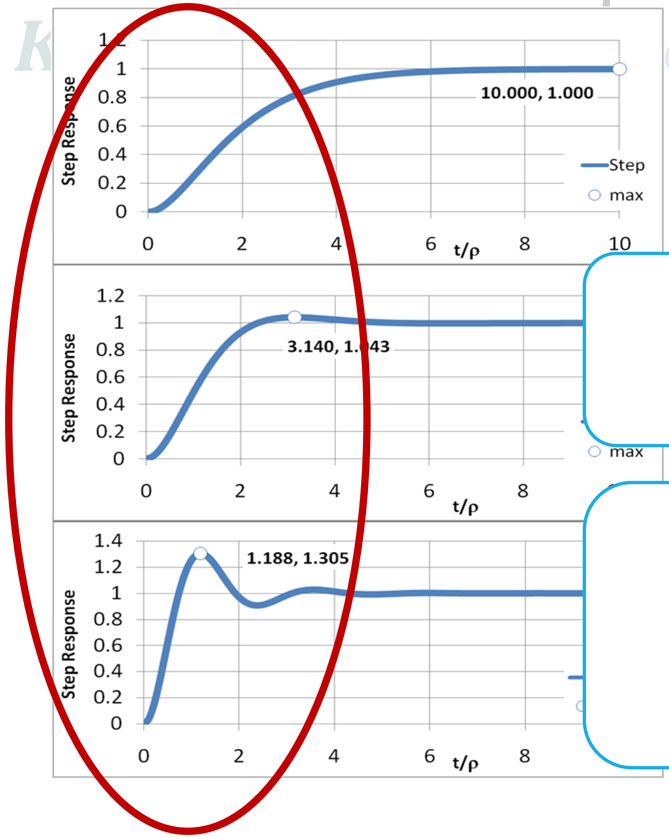
if y(t) = t, differentiate $\frac{dy(t)}{dt} = 1$

if y(t) = t^2, differentiate $\frac{dy(t)}{dt} = 2t$

PID CONTROLLER GENERIC DESIGN

- Analog PID Implementation:

$$G_c(t) = K \quad \int^{t} \quad e(\tau)d\tau + \boxed{K_D * \frac{de(t)}{dt}}$$

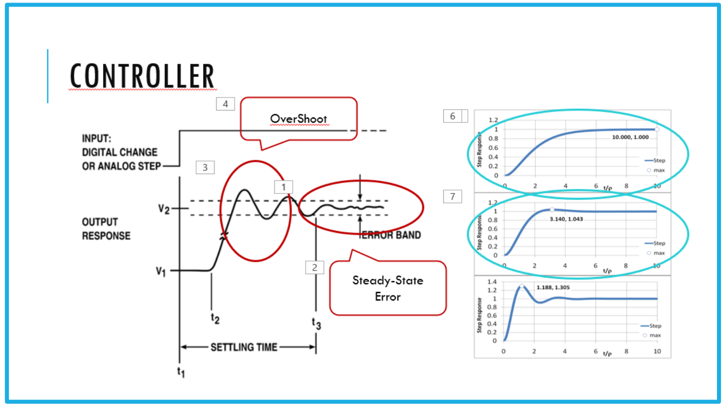


Effect on Steady-state portion of response

minimal

Effect on transient portion of response

Slowing Down the Response, Reduce Overshoot


CONTROLLER
INPUT: DIGITAL CHANGE OR ANALOG STEP
OUTPUT RESPONSE
OverShoot
ERROR BAND
Steady-State Error
SETTLING TIME
V_2
V_1
t_1
t_2
t_3
1
2
3
4
6
7

PID CONTROLLER GENERIC DESIGN

- Analog PID Implementation:

$$G_c(t) = K_P * e(t) + \boxed{K_I * \int_0^t e(\tau)d\tau} + K_D * \frac{de(t)}{dt}$$

Integration :

$$K_I * \int_0^t y(\tau)d\tau$$

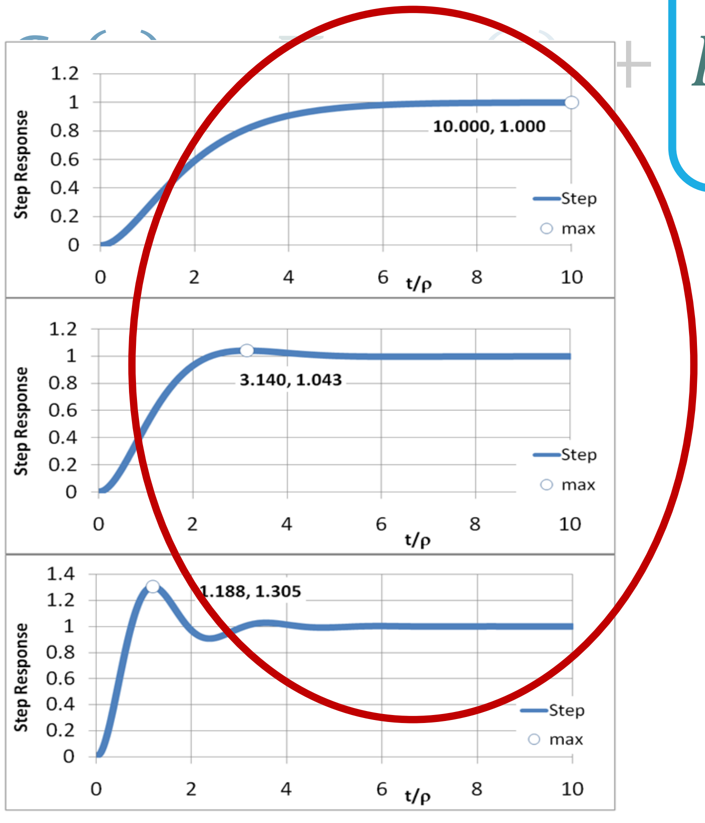
if y(t) = t, integrate $\int_0^t y(\tau)d\tau = t^2$

if y(t) = 1, integrate $\int_0^t y(\tau)d\tau = t$

PID CONTROLLER GENERIC DESIGN

- Analog PID Implementation:

$$+ K_I * \int_0^t e(\tau)d\tau + K_D * \frac{de(t)}{dt}$$



Effect on Steady-state portion of response

Reduce the steady-state Error

Effect on transient portion of response

A proper design have to reduce the integral part at transient portion

P,PI,PD,PID CONTROLLERS (PROS/CONS)

What will happen when we increase/decrease Kp, Ki and Kd ?

Closed-loop Response	**Rise Time**	**Overshoot**	**Settling time**	**Steady State Error**
K_p increase	Decrease ✓	Increase ✗	Small change ✓	Decrease
K_i increase	Decrease	Increase ✗	Increase ✗	Eliminate ✓
K_d increase	Small change	decrease ✓	decrease ✓	Small change ✗